Lecture 11 - Clustering

In **supervised learning** data is given in the form (x, y), where $y = f(x)$. In **unsupervised learning** data is given as just x. **Cluster**: a collection of objects that are: similar to each other in the same cluster, dissimilar to objects in other clusters. Clustering is never 'right' or 'wrong', different clusters give different insights on the data.

Types of Clusters

Flat Clustering: given a data set, create clusters and return the clusters.

Hierarchical Clustering: given a data set, create clusters, and create clusters of the clusters, etc. until you have one cluster that is the entire data set.

Hard / Disjunctive Clustering: each item is a member of one cluster.

Soft Clustering: each item has a probability of being part of each cluster.

Overlapping Clustering: an item can be in many clusters at the same time.

Exhaustive Clustering: each item is a member of a cluster.

Partial Clustering: some items do not belong to a cluster (in practise this is just exhaustive clustering with singleton clusters).

The Clustering Problem

Let $X = (x_1, x_2, \ldots, x_d)$ be a d-dimensional feature vector. Let $D = \{X_1, X_2, \ldots, X_n\}$ be a set of vectors. Given data D, group the n vectors into k groups such that the grouping is **optimal**.

A clustering is good if it produces clusters with high **intra-class similarity** and low **inter-class similarity**.

Can use different metrics for determining how similar vectors are (e.g. distance, L_n, etc.), just be careful that the metric satisfies the metric requirements!

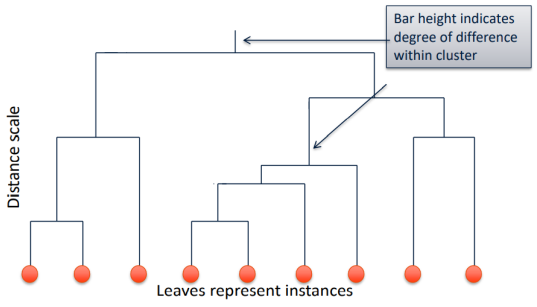
Hierarchical Clustering

Can be done both **top-down** or **bottom-up** (bad time complexity, cannot reverse actions). We maintain a matrix of distance scores for all pairs of:

- Instances

- Clusters (formed so far)

- Instances and Clusters



Bottom-Up Hierarchical Clustering:

for (i = 1 to n) $c_i = \{x_i\}$
$C = \{c_1, \ldots, c_n\}$
j = n
while |C| > 1
    j = j+1
    $(c_a, c_b) = \text{argmin}_{u,v}\ \text{dist}(c_u, c_v)$
    $c_j = c_a \cup c_v$
    add node to tree joining a and b
    $C = C - \{c_a, c_b\} \cup c_j$
return tree with root node j

Divisive / Top-Down Hierarchical Clustering:

Initialize: All items one cluster

Iterate:
1. select a cluster c_j (least coherent)
2. divide c_j into two clusters

Halt: When reaches required # of clusters

Note: Step 2 requires another clustering algorithm!

Partitioning Alg.

Given a *k* and a data set *D* of *n* objects, construct a partitioning of *D* into a set of *k* clusters that optimize the choosen criterion. This algorithm is globally optimal and usually represents the clusters through its centers.

Distance Between Clusters

- **SIngle Link:** distance of the two <u>most</u> similar instances.

$$dist(c_u, c_v) = min\{dist(a, b) | a \in c_u, b \in c_v\}$$

- **Complete Link:** distance of the two <u>least</u> similar instances.

$$dist(c_u, c_v) = max\{dist(a, b) | a \in c_u, b \in c_v\}$$

- **Average Link:** average distance between instances.

$$dist(c_u, c_v) = avg\{dist(a, b) | a \in c_u, b \in c_v\}$$

Conceptual Clustering

Return a description of groups in some language *L*.

k-Means

1) Initialize *k* random centers (either repeat many times or use heuristic for seeds for optimal result)

2) Assign each point to its closest center

3) Move each center to the mean of its points

4) Repeat until convergence

k-Means is O(iterations * clusters * data set size * no. of features). Its weakness is that it is only effective for defined data, needs a *k* defined in advance, struggles with noice, and can only identify clusters in the form of a circle.

Expectation Maximisation Algorithm

Called the **EM Algorithm**. It is a "soft clustering" version of *k*-means, where each cluster is represented by a distribution. EM clustering will converge to a local maximum.

Initialization: choose means of all clusters at random

E-Step: for all points and means, compute $p(point|mean)$ (determine how likely it is that each cluster generates a new instance)

M-Step: set each mean to the *weighted* average of points, with weight $\frac{p(mean)p(point|mean)}{p(point)}$ (adjust cluster parameters to maximize likelihood)

Model-Based Clustering

Build a model that predicts cluster memberships. **Generative Model (e.g. Gaussian cluster):** probability of selecting a cluster, probability of generating an object in cluster etc. **Descriptive Model (e.g. Decision Tree with clusters as leaves).**